CPE 486/586: Machine Learning for Engineers

12 Support Vector Machine

Fall 2026

Rahul Bhadani

Outline

1. Constraint Optimization with Inequality
2. Support Vector Machine
3. Nonlinear Support Vector Machine and Kernel
4. Soft Margin: Allowing Margin Violations

Constraint Optimization with Inequality

Inequality Constraints

$$\min_x f(x)$$
$$\text{subject to}$$
$$g(x) \geq 0$$

Without the constraint, $\frac{df(x^*)}{dx} = 0$.

With the inequality constraint, the solution lies either on the boundary, $g(x^*) = 0$ (the constraint is *active*), or in the interior, $g(x^*) > 0$ (the constraint is *inactive*, and $\frac{df(x^*)}{dx} = 0$ still holds).

Inequality Constraints with Lagrange Multiplier

We could still write the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda^{\mathsf{T}} g(x)$$

$$\frac{\partial \mathcal{L}}{\partial x}(x^*, \lambda^*) = \frac{\partial f}{\partial x}(x^*) - \left(\frac{\partial g}{\partial x}(x^*)\right)^{\mathsf{T}} \lambda^* = 0 \quad \text{(stationarity)} \tag{1}$$

$$g(x^*) \geq 0 \quad \text{(primal feasibility)}$$

$$\lambda^* \geq 0 \quad \text{(dual feasibility)}$$

$$\lambda^* \odot g(x^*) = 0 \quad \text{(complementary slackness)}$$

where $\odot$ is the element-wise product.

These conditions are called **Karush-Kuhn-Tucker Conditions** (KKT).

The Generalized Lagrangian

The generalized Lagrangian function to minimize $f(x)$ subject to equality constraints $g_i(x) = 0$ and inequality constraints $h_j(x) \leq 0$ is

$$L(x, \lambda, \beta) = f(x) + \sum_{i=1}^{I} \lambda_i g_i(x) + \sum_{j=1}^{J} \beta_j h_j(x), \quad \beta_j \geq 0 \tag{2}$$

Karush-Kuhn-Tucker Conditions

- If we have several equality and inequality constraints, in the following form

$$\arg\min_{x} f(x), \text{ s.t. } g_i(x) = 0, h_j(x) \leq 0 \tag{3}$$

The KKT conditions are necessary for optimality (under a constraint qualification). They are also sufficient when f and h_j are convex and g_i are affine:

$$\frac{\partial L(x^*, \lambda^*, \beta^*)}{\partial x} = 0, \quad \frac{\partial L(x^*, \lambda^*, \beta^*)}{\partial \beta} = 0, \quad g_i(x^*) = 0, \quad h_j(x^*) \leq 0,$$
$$\sum_{j=1}^{J} \beta_j^* h_j(x^*) = 0, \quad \beta_j^* \geq 0 \ \ (\lambda_i^* \text{ unrestricted in sign}) \tag{4}$$

Since the terms $\beta_j^* h_j(x^*)$ are nonpositive, $\sum_j \beta_j^* h_j(x^*) = 0$ is equivalent to $\beta_j^* h_j(x^*) = 0$ for $j = 1, \ldots, J$.

The Generalized Lagrangian

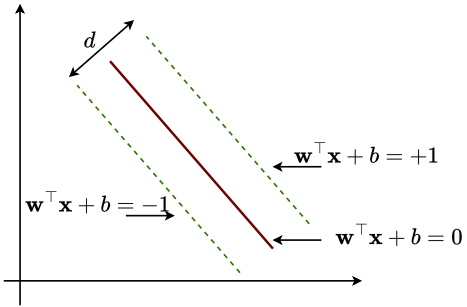
The generalized Lagrangian function to minimize $f(x)$ subject to the equality constraints on $g_i(x)$ and inequality constraints on $h_j(x)$ is given by:

$$L(x, \lambda, \beta) = f(x) + \sum_{i=1}^{I} \lambda_i g_i(x) + \sum_{j=1}^{J} \beta_j h_j(x) \tag{5}$$



Support Vector Machine

Motivating SVM



A linear support vector machine (SVM) aims to find a decision hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximizes the margin of separation.

Assume that all data points satisfy the constraints:

- $\mathbf{w}^\top \mathbf{x}_i + b \geq +1$ where $y_i = +1$
- $\mathbf{w}^\top \mathbf{x}_i + b \leq -1$ where $y_i = -1$

Why ± 1? The Canonical Hyperplane

Scaling $(\mathbf{w}, b)$ by a constant $c > 0$ leaves the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ unchanged.

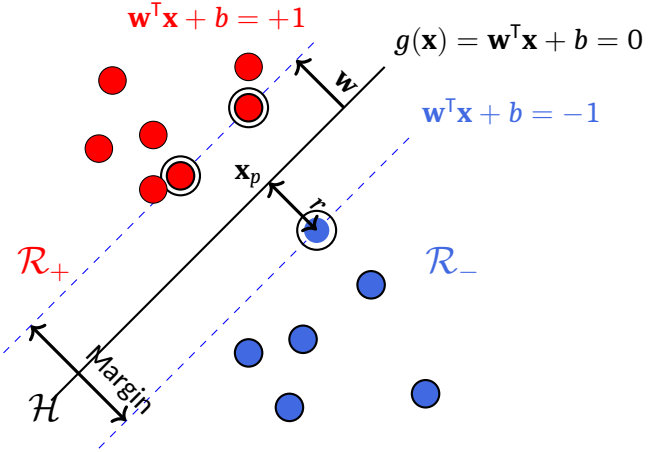
We remove this freedom by fixing the scale so that the closest points satisfy

$$\min_i \; y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1 \tag{6}$$

The training points then satisfy $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$, and the closest points of the two classes lie on $\mathbf{w}^\top \mathbf{x} + b = +1$ and $\mathbf{w}^\top \mathbf{x} + b = -1$.

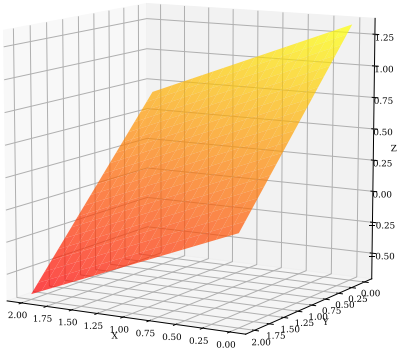
The two bullet constraints on the previous slide collapse into the single constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$.

The SVM and Linear Classification



Equation for a Hyperplane

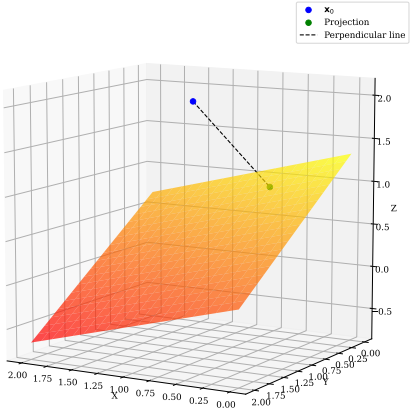
$$\mathbf{w}^\top \mathbf{x} = \beta$$



Euclidean Projection of a Point on a Hyperplane

We project a point $\mathbf{x}_0$ onto the hyperplane $\mathbf{w}^\top \mathbf{x} = \beta$.
The projection point $\mathbf{x}^*$ on the hyperplane is given by

$$\mathbf{x}^* = \mathbf{x}_0 + (\beta - \mathbf{w}^\top \mathbf{x}_0) \frac{\mathbf{w}}{||\mathbf{w}||^2} \qquad (7)$$



Euclidean Projection as an Optimization Problem

One way to derive the above equation is to solve

$$\min_{\mathbf{x}} \frac{1}{2}||\mathbf{x} - \mathbf{x}_0||^2$$

subject to

$$\mathbf{w}^\top \mathbf{x} = \beta$$

Solving the Projection with a Lagrange Multiplier

$$
\begin{aligned}
\mathcal{L}(\mathbf{x}, \lambda) &= \frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|^2 + \lambda(\mathbf{w}^\top \mathbf{x} - \beta) \\
\frac{\partial \mathcal{L}}{\partial \mathbf{x}} &= \mathbf{x} - \mathbf{x}_0 + \lambda \mathbf{w} = 0 \;\Rightarrow\; \mathbf{x} = \mathbf{x}_0 - \lambda \mathbf{w} \\
\mathbf{w}^\top(\mathbf{x}_0 - \lambda \mathbf{w}) = \beta \;&\Rightarrow\; \lambda = \frac{\mathbf{w}^\top \mathbf{x}_0 - \beta}{\|\mathbf{w}\|^2} \\
\mathbf{x}^* &= \mathbf{x}_0 + (\beta - \mathbf{w}^\top \mathbf{x}_0)\frac{\mathbf{w}}{\|\mathbf{w}\|^2}
\end{aligned}
\tag{8}
$$

The distance from $\mathbf{x}_0$ to the hyperplane is

$$
\|\mathbf{x}^* - \mathbf{x}_0\| = \frac{|\mathbf{w}^\top \mathbf{x}_0 - \beta|}{\|\mathbf{w}\|} \tag{9}
$$

With $\beta = -b$, this is $r = |g(\mathbf{x}_0)|/\|\mathbf{w}\|$, the distance r in the SVM figure.

Distance between two Hyperplanes

Consider two parallel hyperplanes:
$\mathcal{H}_1 : \mathbf{w}^\top \mathbf{x} = b_1$
$\mathcal{H}_2 : \mathbf{w}^\top \mathbf{x} = b_2$
We compute the distance between them by projecting a point $\mathbf{x}_1$ on $\mathcal{H}_1$ onto $\mathcal{H}_2$.

$$\mathbf{x}_2 = \mathbf{x}_1 + (b_2 - \mathbf{w}^\top \mathbf{x}_1) \frac{\mathbf{w}}{||\mathbf{w}||^2}$$
$$\mathbf{x}_2 = \mathbf{x}_1 + (b_2 - b_1) \frac{\mathbf{w}}{||\mathbf{w}||^2} \quad (\text{since } \mathbf{w}^\top \mathbf{x}_1 = b_1) \tag{10}$$

Distance between two Hyperplanes

Hence, the distance between two hyperplanes is given by

$$d = ||\mathbf{x}_2 - \mathbf{x}_1|| = \left|\left|(b_2 - b_1)\frac{\mathbf{w}}{||\mathbf{w}||^2}\right|\right|$$

$$d = ||\mathbf{x}_2 - \mathbf{x}_1|| = \frac{|b_2 - b_1|}{||\mathbf{w}||} \tag{11}$$

Formulating SVM Problem: Hyperplane Distance

In the SVM problem, the two margin hyperplanes are

- $\mathbf{w}^\top \mathbf{x} + b = +1$
- $\mathbf{w}^\top \mathbf{x} + b = -1$

Let $b_1 = 1 - b$ and $b_2 = -1 - b$.

Then,

$$d = ||\mathbf{x}_2 - \mathbf{x}_1|| = \frac{|b_2 - b_1|}{||\mathbf{w}||}$$
$$= \frac{|-1 - b - 1 + b|}{||\mathbf{w}||} = \frac{2}{||\mathbf{w}||} \quad (12)$$

Formulating SVM Problem: Objective

We want to maximize d, which is equivalent to minimizing $\frac{||\mathbf{w}||}{2}$.

Both $||\mathbf{w}||$ and $\frac{||\mathbf{w}||^2}{2}$ are convex, and they share the same minimizer. We minimize $\frac{||\mathbf{w}||^2}{2}$ because it is differentiable everywhere (its gradient is $\mathbf{w}$) and turns the problem into a quadratic program.

Final Optimization Problem for SVM

Our discussion of constrained optimization used minimization. The SVM goal of maximizing $d = 2/||\mathbf{w}||$ becomes the following minimization:

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w}, b} \frac{1}{2}\|\mathbf{w}\|_2^2$$

$$\text{s.t. } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, n \quad \% \; 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq 0$$

Lagrangian for SVM

The Lagrangian function for SVM is given by

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2}\|\mathbf{w}\|_2^2 - \sum_{i=1}^{n} \alpha_i \left[y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \right] \tag{13}$$

where $\alpha_i \geq 0$ are the Lagrange multipliers.

From the Primal to the Dual Problem

- **Primal:** $\min_{\mathbf{w},b} \max_{\alpha \geq 0} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner maximum is $+\infty$ when a constraint is violated and $\frac{1}{2}\|\mathbf{w}\|^2$ otherwise, so this is the original problem.

From the Primal to the Dual Problem

- **Primal:** $\min_{\mathbf{w},b} \max_{\alpha \geq 0} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner maximum is $+\infty$ when a constraint is violated and $\frac{1}{2}\|\mathbf{w}\|^2$ otherwise, so this is the original problem.
- **Dual:** swap the order, $\max_{\alpha \geq 0} \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner minimum is what we compute next, from $\partial\mathcal{L}/\partial\mathbf{w} = 0$ and $\partial\mathcal{L}/\partial b = 0$.

From the Primal to the Dual Problem

- **Primal:** $\min_{\mathbf{w},b} \max_{\alpha \geq 0} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner maximum is $+\infty$ when a constraint is violated and $\frac{1}{2}\|\mathbf{w}\|^2$ otherwise, so this is the original problem.
- **Dual:** swap the order, $\max_{\alpha \geq 0} \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner minimum is what we compute next, from $\partial\mathcal{L}/\partial\mathbf{w} = 0$ and $\partial\mathcal{L}/\partial b = 0$.
- **Weak duality:** dual optimum $\leq$ primal optimum.

From the Primal to the Dual Problem

- **Primal:** $\min_{\mathbf{w},b} \max_{\alpha \geq 0} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner maximum is $+\infty$ when a constraint is violated and $\frac{1}{2}\|\mathbf{w}\|^2$ otherwise, so this is the original problem.
- **Dual:** swap the order, $\max_{\alpha \geq 0} \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b, \alpha)$. The inner minimum is what we compute next, from $\partial \mathcal{L}/\partial \mathbf{w} = 0$ and $\partial \mathcal{L}/\partial b = 0$.
- **Weak duality:** dual optimum $\leq$ primal optimum.
- **Strong duality:** the SVM primal is a convex QP with affine constraints, so the two optima are equal when the data are separable. Solving the dual for α^* therefore gives the primal solution $\mathbf{w}^*$.

Deriving the Support Vector Machine

For fixed α, we find the $\mathbf{w}$ and b that minimize $\mathcal{L}$. Setting the derivative of $\mathcal{L}$ w.r.t. $\mathbf{w}$ equal to zero gives

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0 \quad \rightarrow \quad \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i$$

Deriving the Support Vector Machine

Setting the gradient equal to zero gave us an expression for $\mathbf{w}$! Next, we set the derivative of $\mathcal{L}$ w.r.t. b to zero:

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \quad \rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Observation: By taking the derivative w.r.t. b, we ended up getting a constraint!

Deriving the Support Vector Machine

We simplify $\mathcal{L}$ by substituting the expression for $\mathbf{w}$ and the constraint $\alpha^\mathsf{T}\mathbf{y} = \sum_i \alpha_i y_i = 0$.

$$\mathcal{L}(\alpha) =$$

Deriving the Support Vector Machine

We simplify $\mathcal{L}$ by substituting the expression for $\mathbf{w}$ and the constraint $\alpha^\mathsf{T}\mathbf{y} = \sum_i \alpha_i y_i = 0$.

$$\mathcal{L}(\alpha) = \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j - \sum_{i=1}^{n}\alpha_i y_i \mathbf{w}^\mathsf{T}\mathbf{x}_i - b\sum_{i=1}^{n}\alpha_i y_i + \sum_{i=1}^{n}\alpha_i$$

Deriving the Support Vector Machine

We simplify $\mathcal{L}$ by substituting the expression for $\mathbf{w}$ and the constraint $\alpha^\mathsf{T}\mathbf{y} = \sum_i \alpha_i y_i = 0$.

$$\mathcal{L}(\alpha) = \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j - \sum_{i=1}^{n}\alpha_i y_i \mathbf{w}^\mathsf{T}\mathbf{x}_i - b\sum_{i=1}^{n}\alpha_i y_i + \sum_{i=1}^{n}\alpha_i$$

$$= \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j - \sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j + \sum_{i=1}^{n}\alpha_i$$

Deriving the Support Vector Machine

We simplify $\mathcal{L}$ by substituting the expression for $\mathbf{w}$ and the constraint $\alpha^\mathsf{T}\mathbf{y} = \sum_i \alpha_i y_i = 0$.

$$\begin{aligned}
\mathcal{L}(\alpha) &= \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j - \sum_{i=1}^{n}\alpha_i y_i \mathbf{w}^\mathsf{T}\mathbf{x}_i - b\sum_{i=1}^{n}\alpha_i y_i + \sum_{i=1}^{n}\alpha_i \\
&= \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j - \sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j + \sum_{i=1}^{n}\alpha_i \\
&= \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{j=1}^{n}\sum_{i=1}^{n}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^\mathsf{T}\mathbf{x}_j
\end{aligned}$$

Summary of the SVM so far

- The expression below is known as the *dual form* of the constrained optimization task, which is maximized over α. Note that $\mathbf{x}_i^\top \mathbf{x}_j$ measures the similarity between two vectors, so we can replace it with a kernel function $\kappa(\mathbf{x}_i, \mathbf{x}_j)$.

$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j), \text{ s.t. } \sum_{i=1}^{n} y_i \alpha_i = 0, \alpha_i \geq 0 \tag{14}$$

Summary of the SVM so far

- The expression below is known as the *dual form* of the constrained optimization task, which is maximized over α. Note that $\mathbf{x}_i^\mathsf{T}\mathbf{x}_j$ measures the similarity between two vectors, so we can replace it with a kernel function $\kappa(\mathbf{x}_i, \mathbf{x}_j)$.

$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j), \text{ s.t. } \sum_{i=1}^{n} y_i \alpha_i = 0, \alpha_i \geq 0 \tag{14}$$

- Let $\Phi(\mathbf{x})$ be a high-dimensional nonlinear representation of $\mathbf{x}$.

$$\mathbf{w}^\mathsf{T}\Phi(\mathbf{x}) + b = \left(\sum_{i=1}^{n} \alpha_i y_i \Phi(\mathbf{x}_i) \right)^\mathsf{T} \Phi(\mathbf{x}) + b = \sum_{i=1}^{n} \alpha_i y_i \Phi(\mathbf{x}_i)^\mathsf{T} \Phi(\mathbf{x}) + b \tag{15}$$

where $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\mathsf{T}\Phi(\mathbf{x}_j)$. The data enter these expressions only through dot products.

Support Vectors from the KKT Conditions

Complementary slackness for the SVM constraints gives

$$\alpha_i^* \left[y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \right] = 0, \quad i = 1, \ldots, n \tag{16}$$

- If $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) > 1$, the point lies beyond the margin, so $\alpha_i^* = 0$.

Support Vectors from the KKT Conditions

Complementary slackness for the SVM constraints gives

$$\alpha_i^* \left[y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \right] = 0, \quad i = 1, \ldots, n \tag{16}$$

- If $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) > 1$, the point lies beyond the margin, so $\alpha_i^* = 0$.
- If $\alpha_i^* > 0$, then $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) = 1$, and the point lies on the margin. These points are the **support vectors**.

Support Vectors from the KKT Conditions

Complementary slackness for the SVM constraints gives

$$\alpha_i^* \left[y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) - 1 \right] = 0, \quad i = 1, \ldots, n \tag{16}$$

- If $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) > 1$, the point lies beyond the margin, so $\alpha_i^* = 0$.
- If $\alpha_i^* > 0$, then $y_i(\mathbf{w}^{*\top}\mathbf{x}_i + b^*) = 1$, and the point lies on the margin. These points are the **support vectors**.

Since $\mathbf{w}^* = \sum_i \alpha_i^* y_i \mathbf{x}_i$, only the support vectors determine the hyperplane. Removing a non-support vector from the training set leaves the solution unchanged.

Recovering b and Classifying a New Point

Pick a support vector $\mathbf{x}_s$ with $\alpha_s^* > 0$. From $y_s(\mathbf{w}^{*\top}\mathbf{x}_s + b^*) = 1$ and $y_s^2 = 1$:

$$b^* = y_s - \sum_{i \in \mathcal{S}} \alpha_i^* y_i \, \kappa(\mathbf{x}_i, \mathbf{x}_s) \tag{17}$$

where $\mathcal{S}$ is the set of support vectors. In practice, average b^* over the support vectors for numerical stability (for the soft margin, use those with $0 < \alpha_s^* < C$).

Decision rule for a new point $\mathbf{x}$:

$$\hat{y} = \operatorname{sign}\left(\sum_{i \in \mathcal{S}} \alpha_i^* y_i \, \kappa(\mathbf{x}_i, \mathbf{x}) + b^* \right) \tag{18}$$

Prediction cost grows with the number of support vectors $|\mathcal{S}|$, which is usually much smaller than n.

Worked Example: Hard Margin SVM by Hand

Data:

- $\mathbf{x}_1 = (3, 2), \quad y_1 = +1$
- $\mathbf{x}_2 = (2, 0), \quad y_2 = +1$
- $\mathbf{x}_3 = (0, 0), \quad y_3 = -1$

Guess from a sketch: $\mathbf{x}_2$ and $\mathbf{x}_3$ are the support vectors, so $\alpha_1 = 0$.

$\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \alpha_2 = \alpha_3 = \alpha.$

Worked Example: Hard Margin SVM by Hand

Data:

- $\mathbf{x}_1 = (3, 2), \quad y_1 = +1$
- $\mathbf{x}_2 = (2, 0), \quad y_2 = +1$
- $\mathbf{x}_3 = (0, 0), \quad y_3 = -1$

Guess from a sketch: $\mathbf{x}_2$ and $\mathbf{x}_3$ are the support vectors, so $\alpha_1 = 0$.

$\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \alpha_2 = \alpha_3 = \alpha.$

$$\mathcal{L}(\alpha) = 2\alpha - \frac{1}{2}\alpha^2 \|\mathbf{x}_2 - \mathbf{x}_3\|^2 = 2\alpha - 2\alpha^2$$

$$\frac{d\mathcal{L}}{d\alpha} = 2 - 4\alpha = 0 \;\Rightarrow\; \alpha = \tfrac{1}{2} \tag{19}$$

$$\mathbf{w}^* = \tfrac{1}{2}\mathbf{x}_2 - \tfrac{1}{2}\mathbf{x}_3 = (1, 0)$$

$$b^* = y_2 - \mathbf{w}^{*\top}\mathbf{x}_2 = 1 - 2 = -1$$

Worked Example: Hard Margin SVM by Hand

Data:

- $\mathbf{x}_1 = (3, 2), \quad y_1 = +1$
- $\mathbf{x}_2 = (2, 0), \quad y_2 = +1$
- $\mathbf{x}_3 = (0, 0), \quad y_3 = -1$

Guess from a sketch: $\mathbf{x}_2$ and $\mathbf{x}_3$ are the support vectors, so $\alpha_1 = 0$.

$\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \alpha_2 = \alpha_3 = \alpha.$

$$\mathcal{L}(\alpha) = 2\alpha - \frac{1}{2}\alpha^2 \|\mathbf{x}_2 - \mathbf{x}_3\|^2 = 2\alpha - 2\alpha^2$$

$$\frac{d\mathcal{L}}{d\alpha} = 2 - 4\alpha = 0 \;\Rightarrow\; \alpha = \tfrac{1}{2} \tag{19}$$

$$\mathbf{w}^* = \tfrac{1}{2}\mathbf{x}_2 - \tfrac{1}{2}\mathbf{x}_3 = (1, 0)$$

$$b^* = y_2 - \mathbf{w}^{*\top}\mathbf{x}_2 = 1 - 2 = -1$$

Boundary: the vertical line through $(1, 0)$. Margin: $2/\|\mathbf{w}^*\| = 2$.

Check $\mathbf{x}_1$: $y_1(\mathbf{w}^{*\top}\mathbf{x}_1 + b^*) = 2 > 1$, so $\alpha_1 = 0$ agrees with the KKT conditions.

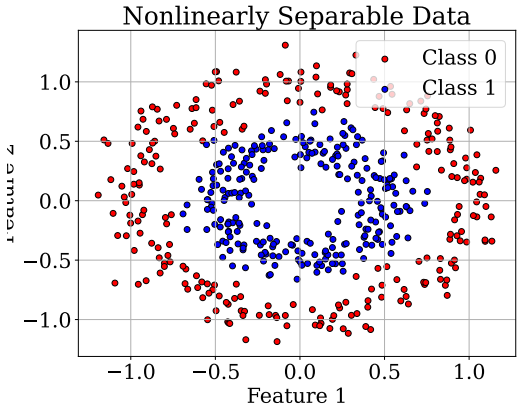
≠ Nonlinear Support Vector Machine and Kernel

Nonlinearly Separable Data

Most real-world problems exhibit data that are not linearly separable.

Nonlinearly Separable Data

How do we separate these two classes?



Kernel for Nonlinear Separation

Solution: map the original features into a higher-dimensional space with a function $\Phi(\mathbf{x})$, called the **feature map**. A **kernel** computes the dot product in that space directly:
$\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\intercal \Phi(\mathbf{x}')$.

What are kernels?

In layman's terms:

What are kernels?

In layman's terms: A kernel is a measure of similarity between two patterns $\mathbf{x}$ and $\mathbf{x}'$

What are kernels?

In layman's terms: A kernel is a measure of similarity between two patterns $\mathbf{x}$ and $\mathbf{x}'$

- Consider a similarity measure of the form:

$$\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$
$$(\mathbf{x}, \mathbf{x}') \mapsto \kappa(\mathbf{x}, \mathbf{x}')$$

What are kernels?

In layman's terms: A kernel is a measure of similarity between two patterns $\mathbf{x}$ and $\mathbf{x}'$

- Consider a similarity measure of the form:

$$\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$
$$(\mathbf{x}, \mathbf{x}') \mapsto \kappa(\mathbf{x}, \mathbf{x}')$$

  - $\kappa(\mathbf{x}, \mathbf{x}')$ returns a real-valued quantity measuring the similarity between $\mathbf{x}$ and $\mathbf{x}'$

What are kernels?

In layman's terms: A kernel is a measure of similarity between two patterns $\mathbf{x}$ and $\mathbf{x}'$

- Consider a similarity measure of the form:

$$\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$
$$(\mathbf{x}, \mathbf{x}') \mapsto \kappa(\mathbf{x}, \mathbf{x}')$$

  - $\kappa(\mathbf{x}, \mathbf{x}')$ returns a real-valued quantity measuring the similarity between $\mathbf{x}$ and $\mathbf{x}'$
  - One simple measure of similarity is the *canonical dot product*
    - computes the cosine of the angle between two vectors, provided they are normalized to length 1
    - a vector space equipped with a dot product is a *pre-Hilbert space* (inner product space)

What are kernels?

In layman's terms: A kernel is a measure of similarity between two patterns $\mathbf{x}$ and $\mathbf{x}'$

- Consider a similarity measure of the form:

$$\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$
$$(\mathbf{x}, \mathbf{x}') \mapsto \kappa(\mathbf{x}, \mathbf{x}')$$

  - $\kappa(\mathbf{x}, \mathbf{x}')$ returns a real-valued quantity measuring the similarity between $\mathbf{x}$ and $\mathbf{x}'$
  - One simple measure of similarity is the *canonical dot product*
    - computes the cosine of the angle between two vectors, provided they are normalized to length 1
    - a vector space equipped with a dot product is a *pre-Hilbert space* (inner product space)

- A feature map Φ **represents** patterns in a dot product space $\mathcal{H}$, where the kernel computes dot products

$$\Phi : \mathcal{X} \to \mathcal{H} \tag{20}$$

Feature Space: $\mathcal{X} \rightarrow \mathcal{H}$

A few notes about our mapping into $\mathcal{H}$ via Φ

1. Mapping lets us define a similarity measure from the dot product in $\mathcal{H}$

$$\kappa(\mathbf{x}, \mathbf{x}') := \Phi(\mathbf{x})^{\mathsf{T}} \Phi(\mathbf{x}') \tag{21}$$

Feature Space: $\mathcal{X} \rightarrow \mathcal{H}$

A few notes about our mapping into $\mathcal{H}$ via Φ

1. Mapping lets us define a similarity measure from the dot product in $\mathcal{H}$

$$\kappa(\mathbf{x}, \mathbf{x}') := \Phi(\mathbf{x})^{\mathsf{T}} \Phi(\mathbf{x}') \tag{21}$$

2. Patterns are dealt with geometrically; efficient learning algorithms may be applied using linear algebra

Feature Space: $\mathcal{X} \rightarrow \mathcal{H}$

A few notes about our mapping into $\mathcal{H}$ via Φ

1. Mapping lets us define a similarity measure from the dot product in $\mathcal{H}$

$$\kappa(\mathbf{x}, \mathbf{x}') := \Phi(\mathbf{x})^{\mathsf{T}} \Phi(\mathbf{x}') \tag{21}$$

2. Patterns are dealt with geometrically; efficient learning algorithms may be applied using linear algebra
3. Different choices of $\Phi(\cdot)$ give different similarity measures

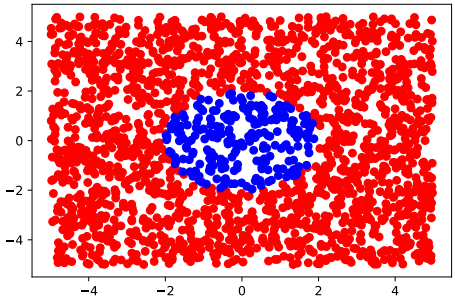
A simple kernel example

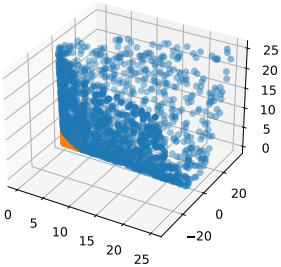
Consider the dot product of two mapped vectors $\mathbf{x}, \mathbf{z} \in \mathbb{R}^2$:

$$\Phi(\mathbf{x})^\top \Phi(\mathbf{z}) = \begin{bmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{bmatrix}^\top \begin{bmatrix} z_1^2 \\ \sqrt{2}z_1z_2 \\ z_2^2 \end{bmatrix} = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 \tag{22}$$

$$= (x_1z_1 + x_2z_2)^2 = (\mathbf{x}^\top \mathbf{z})^2 = \kappa(\mathbf{x}, \mathbf{z})$$

A simple kernel example





Gaussian Kernel

The Gaussian kernel is defined as

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\{(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)\} \tag{23}$$

The term in the exponent $(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ is a scalar value computed in the vector space of the data. Recall the dot product for two arbitrary vectors is given by

$$\mathbf{x}^T\mathbf{x}' = \sum_{i=1}^{d} x_i x_i' \tag{24}$$

Expanding $\|\mathbf{x} - \mathbf{x}'\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2\mathbf{x}^\mathsf{T}\mathbf{x}'$ exposes a dot product:

$$\kappa(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}\|^2} e^{-\gamma\|\mathbf{x}'\|^2} e^{2\gamma\mathbf{x}^\mathsf{T}\mathbf{x}'} = e^{-\gamma\|\mathbf{x}\|^2} e^{-\gamma\|\mathbf{x}'\|^2} \sum_{n=0}^{\infty} \frac{(2\gamma)^n}{n!} \left(\mathbf{x}^\mathsf{T}\mathbf{x}'\right)^n \tag{25}$$

The term $(\mathbf{x}^\mathsf{T}\mathbf{x}')^n$ is a polynomial kernel with a finite feature map. The infinite sum therefore corresponds to an infinite-dimensional Φ.

The Kernel Trick

- The "trick" in each of the kernel examples is that we never needed to calculate $\Phi(\mathbf{x})$. Instead, we applied a nonlinear function to a dot product and this was equivalent to the dot product in a high dimensional space.
  - *This is known as the kernel trick!*
  - Kernels allow us to solve nonlinear classification problems by taking advantage of Cover's Theorem without the overhead of finding the high dimensional space.

Cover's Theorem

Cover (1965)

A pattern classification problem cast nonlinearly into a high-dimensional space is more likely to be linearly separable than in a low-dimensional space.

- For n points in general position in $\mathbb{R}^d$, the fraction of the 2^n labelings that a hyperplane can separate is

$$P(n,d) = \frac{1}{2^{n-1}} \sum_{m=0}^{d} \binom{n-1}{m} \tag{26}$$

- $P(n,d) = 1$ when $n \leq d+1$. Example: $P(4,2) = 7/8$, since XOR cannot be separated in $\mathbb{R}^2$.
- Raising d through Φ raises the chance of separability. The kernel trick gives us that space without computing Φ.

Which Functions Are Valid Kernels?

- κ is a valid kernel when a feature map Φ exists with $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\intercal \Phi(\mathbf{x}')$.

Which Functions Are Valid Kernels?

- κ is a valid kernel when a feature map Φ exists with $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\intercal \Phi(\mathbf{x}')$.
- **Mercer's condition:** κ is symmetric, and the Gram matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite for any finite set of points.

Which Functions Are Valid Kernels?

- κ is a valid kernel when a feature map Φ exists with $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\mathsf{T}\Phi(\mathbf{x}')$.
- **Mercer's condition:** κ is symmetric, and the Gram matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite for any finite set of points.
- A PSD Gram matrix keeps the dual a convex QP, so the solver reaches the global optimum.

Which Functions Are Valid Kernels?

- κ is a valid kernel when a feature map Φ exists with $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\mathsf{T}\Phi(\mathbf{x}')$.
- **Mercer's condition:** κ is symmetric, and the Gram matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite for any finite set of points.
- A PSD Gram matrix keeps the dual a convex QP, so the solver reaches the global optimum.
- Linear, polynomial ($c \geq 0$), and Gaussian kernels satisfy Mercer's condition. The $\tanh$ kernel fails it for some (γ, c), so its Gram matrix can have negative eigenvalues.

Which Functions Are Valid Kernels?

- κ is a valid kernel when a feature map Φ exists with $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\mathsf{T}\Phi(\mathbf{x}')$.
- **Mercer's condition:** κ is symmetric, and the Gram matrix $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite for any finite set of points.
- A PSD Gram matrix keeps the dual a convex QP, so the solver reaches the global optimum.
- Linear, polynomial ($c \geq 0$), and Gaussian kernels satisfy Mercer's condition. The $\tanh$ kernel fails it for some (γ, c), so its Gram matrix can have negative eigenvalues.
- Sums and products of valid kernels are valid kernels.

Other popular kernel choices

Radial Basis Function (RBF) Kernel (the Gaussian kernel)

$$\kappa(\mathbf{x}, \mathbf{z}) = \exp\{(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)\} \tag{27}$$

Polynomial Kernel

$$\kappa(\mathbf{x}, \mathbf{z}) = (\gamma\, \mathbf{x}^\mathsf{T}\mathbf{z} + c)^p \tag{28}$$

Hyperbolic Tangent Kernel

$$\kappa(\mathbf{x}, \mathbf{z}) = \tanh\big(\gamma\, \mathbf{x}^\mathsf{T}\mathbf{z} + c\big) \tag{29}$$

Soft Margin: Allowing Margin Violations

Soft Margin for SVM

Some data sets cannot be separated by a linear hyperplane because of outliers, noise, or weak nonlinearity. To keep a linear boundary, we let some points violate the margin or be misclassified, at a cost.

Slack Variable

We introduce a slack variable $\xi_i \geq 0$ for sample i such that

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$$

Soft Margin Optimization Problem

A New Formulation

We can modify the objective of maximizing the margin by introducing *slack variables* to meet the constraints

$$\begin{aligned} &\arg\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i, \\ &\text{s.t. } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geqslant 1 - \xi_i, \quad \xi_i \geqslant 0 \end{aligned} \tag{30}$$

where $\xi_i > 0$ only for samples that violate the margin, and $C > 0$ sets the price of a violation (large C approaches the hard margin).

Deriving the Soft Margin SVM

The Lagrangian function is given by

$$L(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\mu_i\xi_i - \sum_{i=1}^{n}\alpha_i\left[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i\right] \quad (31)$$

where $\alpha_i \geq 0$ and $\mu_i \geq 0$ are Lagrange multipliers.

Deriving the Soft Margin SVM

The Lagrangian function is given by

$$L(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\mu_i\xi_i - \sum_{i=1}^{n}\alpha_i\left[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i\right] \quad (31)$$

where $\alpha_i \geq 0$ and $\mu_i \geq 0$ are Lagrange multipliers.

Our approach is the same as it was with the hard margin SVM. We have the Lagrangian function and we're going to find the dual form by computing

$$\frac{\partial L}{\partial \mathbf{w}}, \quad \frac{\partial L}{\partial b}, \quad \frac{\partial L}{\partial \xi_i} \quad (32)$$

Deriving the Soft Margin SVM

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0 \longrightarrow \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i \quad \text{\% same result as before} \tag{33}$$

Substitute these results back into L. The ξ_i terms cancel because $\alpha_i + \mu_i = C$.

Deriving the Soft Margin SVM

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0 \longrightarrow \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i \quad \% \text{ same result as before} \tag{33}$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \longrightarrow C = \alpha_i + \mu_i \quad \% \ 0 \leq \alpha_i \leq C \text{ since } \mu_i \geq 0 \tag{34}$$

Substitute these results back into L. The ξ_i terms cancel because $\alpha_i + \mu_i = C$.

Deriving the Soft Margin SVM

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0 \longrightarrow \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i \quad \% \text{ same result as before} \tag{33}$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \longrightarrow C = \alpha_i + \mu_i \quad \% \ 0 \leq \alpha_i \leq C \text{ since } \mu_i \geq 0 \tag{34}$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \longrightarrow \sum_{i=1}^{n} \alpha_i y_i = 0 \quad \% \text{ same result as before} \tag{35}$$

Substitute these results back into L. The ξ_i terms cancel because $\alpha_i + \mu_i = C$.

Deriving the Soft Margin SVM

$$L(\alpha) = \tag{36}$$

Deriving the Soft Margin SVM

$$L(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \xi_i - \sum_{i=1}^{n} \mu_i \xi_i \tag{36}$$

Deriving the Soft Margin SVM

$$
\begin{aligned}
L(\alpha) &= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \xi_i - \sum_{i=1}^{n} \mu_i \xi_i \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \xi_i (\alpha_i + \mu_i)
\end{aligned} \tag{36}
$$

Deriving the Soft Margin SVM

$$
\begin{aligned}
L(\alpha) &= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \xi_i - \sum_{i=1}^{n} \mu_i \xi_i \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \xi_i (\alpha_i + \mu_i) \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - C \sum_{i=1}^{n} \xi_i
\end{aligned}
\tag{36}
$$

Deriving the Soft Margin SVM

$$
\begin{aligned}
L(\alpha) &= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \xi_i - \sum_{i=1}^{n} \mu_i \xi_i \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \xi_i (\alpha_i + \mu_i) \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j + C \sum_{i=1}^{n} \xi_i - C \sum_{i=1}^{n} \xi_i \\
&= \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{\% same result as the hard margin}
\end{aligned}
\tag{36}
$$

The Soft Margin SVM

$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j), \text{ s.t. } \sum_{i=1}^{n} y_i \alpha_i = 0, 0 \leq \alpha_i \leq C \tag{37}$$

- The soft margin dual matches the hard margin dual except for one change: the Lagrange multiplier α_i is now bounded above by C (a *box constraint*).

The Soft Margin SVM

$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j), \text{ s.t. } \sum_{i=1}^{n} y_i \alpha_i = 0, 0 \leq \alpha_i \leq C \tag{37}$$

- The soft margin dual matches the hard margin dual except for one change: the Lagrange multiplier α_i is now bounded above by C (a *box constraint*).
- The optimization problem might look difficult; however, this is a *quadratic program* and there are optimizers available to solve this problem. See `CVXOPT` or `CVXPY` in Python. The `SVC` class in `scikit-learn` solves it with an SMO-type algorithm (`libsvm`).

The Soft Margin SVM

$$\arg\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{j=1}^{n} \sum_{i=1}^{n} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j), \text{ s.t. } \sum_{i=1}^{n} y_i \alpha_i = 0, 0 \leq \alpha_i \leq C \tag{37}$$

- The soft margin dual matches the hard margin dual except for one change: the Lagrange multiplier α_i is now bounded above by C (a *box constraint*).
- The optimization problem might look difficult; however, this is a *quadratic program* and there are optimizers available to solve this problem. See `CVXOPT` or `CVXPY` in Python. The `SVC` class in `scikit-learn` solves it with an SMO-type algorithm (`libsvm`).
- Many α_i will be zero after solving the QP. The data samples that correspond to non-zero values of α_i are known as the *support vectors*.

Interpreting α_i and ξ_i in the Soft Margin SVM

KKT conditions: $\alpha_i \left[y_i f(\mathbf{x}_i) - 1 + \xi_i \right] = 0$ and $\mu_i \xi_i = (C - \alpha_i)\, \xi_i = 0$, with $f(\mathbf{x}) = \mathbf{w}^\mathsf{T}\mathbf{x} + b$.

α_i	Condition	Location of $\mathbf{x}_i$
$\alpha_i = 0$	$y_i f(\mathbf{x}_i) \geq 1, \quad \xi_i = 0$	correct side, on or beyond the margin
$0 < \alpha_i < C$	$y_i f(\mathbf{x}_i) = 1, \quad \xi_i = 0$	exactly on the margin
$\alpha_i = C$	$y_i f(\mathbf{x}_i) = 1 - \xi_i$	inside the margin ($0 < \xi_i \leq 1$) or misclassified ($\xi_i > 1$)

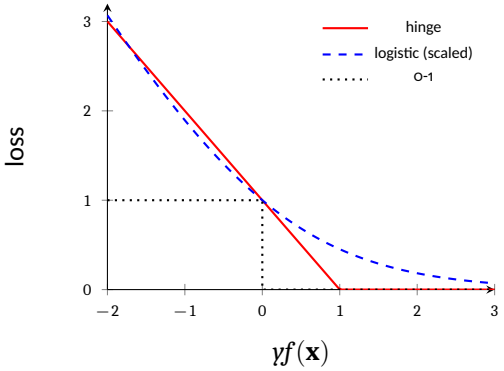
Support vectors are the points with $\alpha_i > 0$ (rows 2 and 3).

Soft Margin SVM as Hinge Loss Minimization

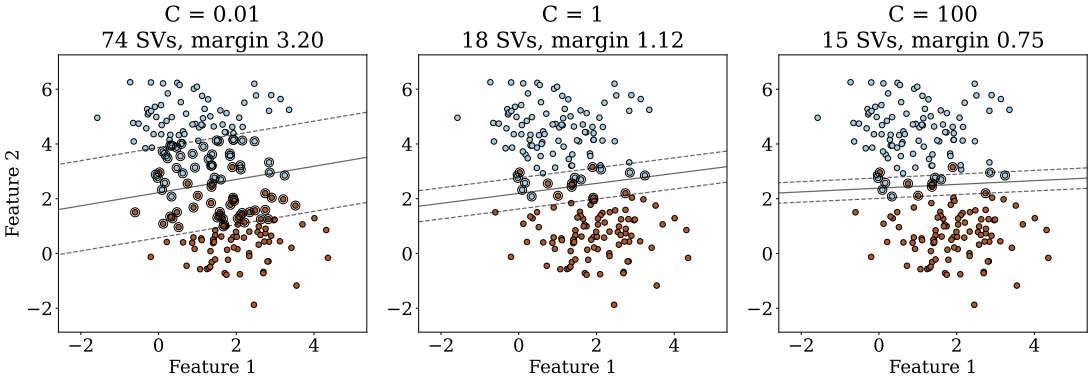
At the optimum, $\xi_i = \max\left(0,\ 1 - y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)\right)$. Substituting into the primal:

$$\min_{\mathbf{w},b}\ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\underbrace{\max\left(0,\ 1 - y_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b)\right)}_{\text{hinge loss}} \tag{38}$$

- Regularized loss minimization, the same template as regularized logistic regression with loss $\log\left(1 + e^{-yf(\mathbf{x})}\right)$.
- Hinge loss is exactly zero for $yf(\mathbf{x}) \geq 1$, which is why many α_i are zero.
- `LinearSVC(loss="hinge")` in the notebook solves this problem (it also regularizes b).

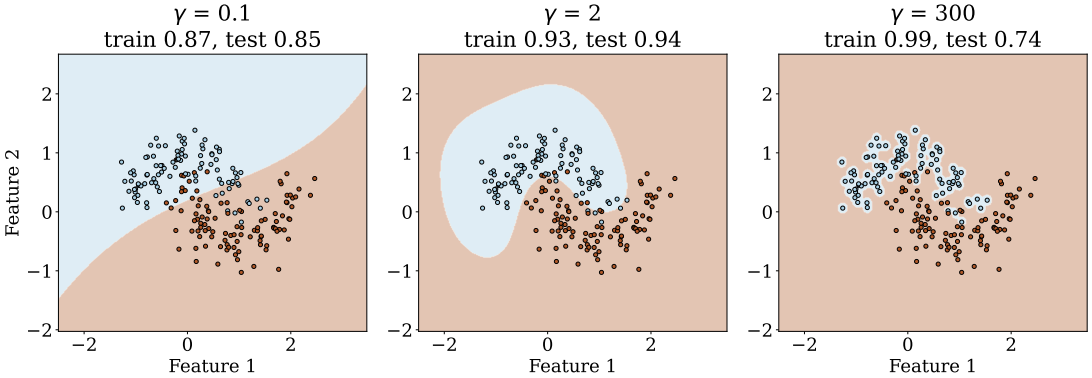


Effect of C (Linear Kernel)



Small C tolerates violations: wide margin, many support vectors. Large C punishes them: narrow margin, close to the hard margin solution.

Effect of γ (RBF Kernel)



γ sets how far the influence of a training point reaches in $\kappa(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$. Small γ underfits. Large γ wraps the boundary around single points: training accuracy rises while test accuracy falls. Tune C and γ together with cross-validation.

Multi-class SVM

The SVM is a binary classifier. For K classes:

- **One-vs-Rest (OvR):** train K classifiers, class k against the rest. Predict the class with the largest $f_k(\mathbf{x})$.

Multi-class SVM

The SVM is a binary classifier. For K classes:

- **One-vs-Rest (OvR):** train K classifiers, class k against the rest. Predict the class with the largest $f_k(\mathbf{x})$.
- **One-vs-One (OvO):** train $K(K-1)/2$ classifiers, one per pair of classes. Predict by majority vote.

Multi-class SVM

The SVM is a binary classifier. For K classes:

- **One-vs-Rest (OvR):** train K classifiers, class k against the rest. Predict the class with the largest $f_k(\mathbf{x})$.
- **One-vs-One (OvO):** train $K(K-1)/2$ classifiers, one per pair of classes. Predict by majority vote.
- `SVC` always trains OvO internally. Its `decision_function_shape` argument only changes the shape of the returned scores. `LinearSVC` uses OvR.

Multi-class SVM

The SVM is a binary classifier. For K classes:

- **One-vs-Rest (OvR):** train K classifiers, class k against the rest. Predict the class with the largest $f_k(\mathbf{x})$.
- **One-vs-One (OvO):** train $K(K-1)/2$ classifiers, one per pair of classes. Predict by majority vote.
- `SVC` always trains OvO internally. Its `decision_function_shape` argument only changes the shape of the returned scores. `LinearSVC` uses OvR.

Notebook example: $K = 6$ modulation types gives 15 pairwise SVMs.

From the Math to `scikit-learn`

Math	`SVC` parameter or attribute
C	`C`
γ in RBF, polynomial, $\tanh$	`gamma`
c and p in the polynomial kernel	`coef0`, `degree`
support vectors $\{\mathbf{x}_i : \alpha_i > 0\}$	`support_vectors_`, `support_`
$\alpha_i y_i$ for the support vectors	`dual_coef_`
b	`intercept_`
$\mathbf{w}$ (linear kernel only)	`coef_`
$\sum_i \alpha_i y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b$	`decision_function(X)`

Kernels depend on distances and dot products, so standardize features first (`StandardScaler`). Training cost grows roughly as n^2 or faster, so use `LinearSVC` or `SGDClassifier` for large n.

References



Christopher Bishop (2007)

Pattern Recognition and Machine Learning

New York, NY: Springer 1st edition.



Vladimir Vapnik (2000)

The Nature of Statistical Learning

New York, NY: Springer 1st edition.



Christopher Burges (1998)

A Tutorial on Support Vector Machines for Pattern Recognition

Data Mining and Knowledge Discovery, pp. 121–167.

Python Notebook

https://github.com/rahulbhadani/CPE486586_FA26/blob/master/Code/CPE486586_Ch12_Support_Vector_Machine.ipynb

The End